

PROGRAMMATION GRAPHIQUE — OpenGL

Pipeline GPU, GLSL Shaders & Rendu 3D — C++ / Jenga / NKWindow / NKMath

UNITÉ	NIVEAU	DURÉE	BUILD
OpenGL 3.3 Core → 4.6	Débutant → Intermédiaire	~120 h — 16 semaines	Jenga 2.0 + glad + NKWindow

PHILOSOPHIE

Ce cours enseigne la programmation graphique GPU via OpenGL 3.3 Core Profile. L'étudiant comprend le pipeline graphique (vertex → rasterisation → fragment) depuis ses fondements mathématiques jusqu'aux shaders GLSL. Jenga gère le build multi-plateforme, NKWindow crée le contexte OpenGL, et NKMath (construite par les étudiants) fournit les vecteurs et matrices. Zéro bibliothèque de rendu externe — glad pour le chargement des fonctions OpenGL, c'est tout.

ÉCOSYSTÈME TECHNIQUE

1. Jenga — Build System

Chaque TP produit un fichier `.jenga` qui déclare le projet, lie glad et NKWindow, et configure les filtres plateforme.

```
# TP OpenGL type - fichier .jenga
from Jenga import *

with workspace('TP_OpenGL_XX', location='.'):
    configurations(['Debug', 'Release'])
    targetoses([TargetOS.WINDOWS, TargetOS.LINUX, TargetOS.MACOS])
    targetarchs([TargetArch.X86_64])

    with project('TP_GL'):
        windowedapp()
        language('C++')
        cppdialect('C++17')
        files(['src/**/*.cpp', 'glad/src/glad.c', 'nkmath/**/*.cpp'])
        includedirs(['include', 'glad/include', 'nkmath',
                    '%{env.NKENTSEU_ROOT}/include'])
        links(['NKWindow', 'NKCore', 'NKContainers'])
        libdirs(['%{env.NKENTSEU_ROOT}/lib'])
        pchheader('pch.h')
        pchsource('src/pch.cpp')

        with filter('system:Windows'):
            links(['opengl32', 'user32', 'gdi32'])
            defines(['NKENTSEU_PLATFORM_WINDOWS'])
        with filter('system:Linux'):
            links(['GL', 'X11', 'pthread', 'dl'])
            defines(['NKENTSEU_PLATFORM_LINUX'])
        with filter('system:macOS'):
            framework('OpenGL')
            defines(['NKENTSEU_PLATFORM_MACOS'])

        with filter('configurations:Debug'):
            defines(['DEBUG', 'GL_DEBUG']); symbols('On'); optimize('Off')
        with filter('configurations:Release'):
            defines(['NDEBUG']); optimize('Speed'); symbols('Off')
```

2. NKWindow — Contexte OpenGL

NKWindow crée la fenêtre native ET le contexte OpenGL (Core Profile 3.3+). Il expose les événements via NkEventSystem. L'étudiant n'a qu'à appeler glad pour charger les pointeurs de fonctions GL.

```
// Initialisation type NKWindow + OpenGL
NkWindowConfig cfg;
cfg.title      = NkString("TP OpenGL");
cfg.width      = 1280; cfg.height = 720;
cfg.glMajor    = 3;    cfg.glMinor = 3;    // Core Profile
cfg.glCoreProfile = true;
```

```
NkWindow window; window.Create(cfg);
window.MakeContextCurrent();

// Charger les fonctions OpenGL via glad
if (!gladLoadGLLoader((GLADloadproc)window.GetProcAddress()))
    return -1; // échec du chargement

// Boucle principale
NkEventSystem events; events.Init(); events.RegisterWindow(&window);
while (window.IsOpen()) {
    events.PollEvents();
    // rendu OpenGL ici
    window.SwapBuffers(); // présente le backbuffer
}
```

3. NKMath — Construite par les Étudiants

Les étudiants construisent NKMath au fil du cours. Elle fournit tous les types mathématiques nécessaires à OpenGL (vecteurs, matrices 4×4, quaternions).

Type NKMath	Contenu	Usage GL	Module
NkVec2f / NkVec2i	Vecteur 2D float/int, opérateurs, dot, normalize	UV coords, screen pos	M-01
NkVec3f	Vecteur 3D, cross, normalize, reflect	Position, normale, RGB	M-02
NkVec4f	Vecteur 4D homogène, xyzw	gl_Position, couleurs	M-03
NkMat4x4	Matrice 4×4, mul, transpose, inverse	MVP matrix	M-03
NkMathUtils	Lerp, clamp, deg2rad, pi, easing	Animations	M-01
NkQuaternion	Quaternion, slerp, toMat4	Rotations 3D	M-07
NkTransform3D	Position+Rotation+Scale → NkMat4x4	Scène graph	M-08
NkColor / NkColorF	RGBA 8-bit et float, conversions	Couleurs shaders	M-01

PRÉREQUIS

C++

- C++ intermédiaire : classes, templates de base, pointeurs, RAII
- Gestion mémoire : new/delete, smart pointers (NkUniquePtr)
- Compilation séparée, linkage, headers

Mathématiques

- Algèbre linéaire : vecteurs 2D/3D, produit scalaire et vectoriel
- Matrices : multiplication, identité — notion d'inverse
- Trigonométrie : sin, cos, tan, atan2, radians
- Géométrie : droites, plans, triangles

Outils

- Compilateur : g++ 11+, clang++ 13+ ou MSVC 2019+
- Jenga : pip install Jenga
- Carte graphique supportant OpenGL 3.3+ (tout GPU post-2010)
- NKWindow configuré (fourni par le professeur)

BIBLIOTHÈQUES AUTORISÉES

glad (chargeur de fonctions OpenGL) | stb_image.h (chargement textures) |
NKWindow (fenêtre + contexte GL) | NKMath (construite par les étudiants) |
INTERDIT : GLM, GLFW, SDL, Qt, SFML pour le rendu ou les maths

VUE D'ENSEMBLE DU PROGRAMME

Sem.	Mod.	Titre	Thème	Durée
1	01	Fondations : GPU, OpenGL & Contexte	Architecture GPU	8h
2	02	Pipeline Graphique & Premier Triangle	VAO/VBO/Shaders	8h
3	03	GLSL & Shaders Programmables	Vertex & Fragment	8h
4	04	Transformations 3D & Matrices MVP	NkMat4x4, NkVec3/4f	10h
5	05	Textures & Sampler Objects	UV, filtrage, mipmaps	10h
6	06	Caméra 3D & Projection	View/Projection matrix	8h
7	07	Éclairage — Phong & PBR Intro	Normales, light models	10h
8	08	Scene Graph & Instancing	Transform, draw calls	8h
9	09	Framebuffers & Post-Processing	FBO, RBO, effets	10h
10	10	Géométrie Avancée & Geometry Shaders	GS, tessellation intro	8h
11	11	Compute Shaders & GPGPU	Calcul GPU, SSBO	10h
12-13	12	Techniques Avancées (Shadows, PBR, IBL)	Ombres, PBR complet	12h
14-16	13	Projet Final	Moteur 3D OpenGL	18h

Objectifs — NKMath : NkVec2f, NkVec2i, NkColor, NkColorF, NkMathUtils

- Comprendre l'architecture GPU et le pipeline fixe vs programmable
- Créer un contexte OpenGL 3.3 Core Profile via NKWindow
- Charger les fonctions GL avec glad
- Effacer l'écran et afficher le résultat (premier rendu GPU)

S01
THÉORIE**Architecture GPU & Historique OpenGL**

- CPU vs GPU : parallélisme massif, shader cores, SIMD à l'échelle matérielle
- Histoire OpenGL : pipeline fixe (1.x) → pipeline programmable (2.0) → Core Profile (3.3)
- OpenGL Core Profile vs Compatibility Profile : pourquoi on utilise Core
- Drivers, ICD (Installable Client Driver), extensions OpenGL
- Contexte OpenGL : état global, thread owner, make current
- glad : loader de pointeurs de fonctions GL — pourquoi il est nécessaire

S02
TP**Setup : NKWindow + glad + Premier Clear**

- Installation : Jenga, glad (génération via glad.dav1d.de ou fourni)
- Fichier .jenga : lier opengl32/GL, inclure glad/include
- Créer le contexte GL 3.3 Core via NkWindowConfig (glMajor/glMinor/glCoreProfile)
- gladLoadGLLoader() : charger les fonctions
- glClearColor() + glClear(GL_COLOR_BUFFER_BIT) + SwapBuffers()
- NkEventSystem : gestion de NkWindowCloseEvent et Escape
- NKMath : NkVec2f, NkColor, NkMathUtils dans namespace nkentseu

Livable : Fenêtre OpenGL 1280×720 avec fond coloré animé (sin du temps)**S03**
THÉORIE**États OpenGL & Debug**

- Modèle à états OpenGL : glEnable/glDisable, glGet*
- glGetError() : détecter les erreurs OpenGL
- GL_DEBUG_OUTPUT (4.3+) : callback d'erreur asynchrone
- Viewport : glViewport(x, y, w, h) — lien avec NKWindow resize
- Scissor test, face culling, depth test — concepts initiaux

LIVRABLE
M01**Fenêtre OpenGL fonctionnelle + NkVec2f/NkColor/NkMathUtils + fond animé + debug GL activé**

Objectifs — NKMath : NkVec3f

- Maîtriser les objets fondamentaux : VAO, VBO, EBO
- Écrire les premiers shaders GLSL (vertex + fragment)
- Dessiner un triangle et un quad colorés

S04
THÉORIE**VAO, VBO, EBO — Données sur le GPU**

- Vertex Buffer Object (VBO) : envoyer des données vertex au GPU (glGenBuffers, glBindBuffer, glBufferData)
- Vertex Array Object (VAO) : mémoriser la description des attributs vertex
- Element Buffer Object (EBO) : index buffer pour réutiliser des vertices
- Attributs vertex : glVertexAttribPointer, glEnableVertexAttribArray
- Usage hints : GL_STATIC_DRAW, GL_DYNAMIC_DRAW, GL_STREAM_DRAW
- NKMath : NkVec3f — position 3D, cross product, reflect

S05
THÉORIE**Vertex & Fragment Shaders GLSL — Bases**

- GLSL : syntaxe, types (vec2, vec3, vec4, mat4, sampler2D...)
- Vertex Shader : in (attributs), out (varyings), gl_Position
- Fragment Shader : in (varyings interpolés), out vec4 FragColor
- glCreateShader, glShaderSource, glCompileShader, glGetShaderInfoLog
- glCreateProgram, glAttachShader, glLinkProgram, glUseProgram
- Uniforms : glGetUniformLocation, glUniform*

S06
TP**TP — Triangle Coloré & Quad Indexé**

- VBO + VAO : envoyer 3 vertices (position + couleur) au GPU
- Shader minimal : pass-through VS + couleur fixe FS
- Smooth shading : couleur en attribut vertex, interpolée automatiquement
- EBO : quad (4 vertices, 6 indices — 2 triangles)
- Classe ShaderProgram : compile, link, setUniform helpers
- Classe Mesh : encapsule VAO/VBO/EBO

Livrable : Triangle RGB + Quad texturé (couleur UV procédurale) dans NKWindow

Objectifs — NKMath : NkVec4f

- Maîtriser GLSL : types, built-ins, structures de contrôle
- Écrire des shaders procéduraux avancés
- Comprendre l'interpolation des varyings et les uniforms

S07
THÉORIE**GLSL Approfondissement**

- Types GLSL : scalaires, vecteurs, matrices, tableaux, structs
- Built-ins vertex : `gl_Position`, `gl_VertexID`, `gl_InstanceID`
- Built-ins fragment : `gl_FragCoord`, `gl_FrontFacing`, `gl_FragDepth`
- Fonctions GLSL : `mix`, `clamp`, `smoothstep`, `fract`, `mod`, `step`, `length`, `dot`, `cross`, `normalize`, `reflect`, `refract`
- Qualificateurs : `in/out/uniform/layout(location=N)`
- Precision qualifiers : `highp`, `mediump`, `lowp` (mobile/ES)
- NKMath : `NkVec4f` — coordonnées homogènes, `rgba`, `xyzw` swizzle

S08
THÉORIE**Shaders Procéduraux & Effets**

- Signed Distance Fields (SDF) : cercle, rectangle, coins arrondis
- Noise procédural : `hash`, `value noise`, `gradient noise` (Perlin-like)
- UV animation : défilement, rotation, déformation
- Shader time-based : uniforme `float u_time` pour les animations
- Uniforms blocks (UBO) : regrouper les uniforms communs

S09
TP**TP — Shaders Avancés**

- Shader SDF : dessiner un cercle parfait avec antialiasing (`smoothstep`)
- Shader procédural : grille, mandelbrot, bruit `value noise`
- Shader animé : vagues sinusoïdales, morphing de formes
- UBO : passer `NkMat4x4` et `NkVec4f` comme uniform block
- Hot-reload de shaders : recompiler sans quitter l'application

Livrable : Galerie de 6+ effets procéduraux GLSL interactifs

Objectifs — NKMath : NkMat4x4, NkTransform3D

- Construire les matrices Model, View, Projection
- Implémenter NkMat4x4 complète dans NKMath
- Comprendre les espaces de coordonnées 3D

S10
THÉORIE**Espaces de Coordonnées 3D**

- Local Space → World Space → View Space → Clip Space → NDC → Screen Space
- Matrice Model (M) : transformer l'objet dans le monde
- Matrice View (V) : caméra — lookAt(eye, center, up)
- Matrice Projection (P) : perspective (FOV, aspect, near, far) ou orthographique
- $gl_Position = P * V * M * vec4(localPos, 1.0)$ — le MVP
- Homogeneous divide (perspective division) : clip space → NDC

S11
THÉORIE**NKMath — NkMat4x4**

- Matrice 4x4 : 16 floats, stockage column-major (compatible OpenGL)
- identity(), translation(), rotation(axis, angle), scale()
- perspective(fovY, aspect, near, far) — calcul frustum
- orthographic(left, right, bottom, top, near, far)
- lookAt(eye, center, up) — construction de la view matrix
- mul(NkMat4x4), mulVec(NkVec4f), transpose(), inverse()
- toFloatArray() : envoyer à glUniformMatrix4fv

S12
TP**TP — Cube 3D Transformé**

- NkMat4x4 complète dans NKMath (column-major pour OpenGL)
- Mesh cube : 8 vertices, 36 indices
- Uniform MVP : glUniformMatrix4fv avec NkMat4x4::toFloatArray()
- Rotation automatique sur les 3 axes (u_time)
- Profondeur : glEnable(GL_DEPTH_TEST), GL_DEPTH_BUFFER_BIT
- NkTransform3D : position + rotation + scale → NkMat4x4

Livrable : Cube 3D coloré tournant avec transformations MVP correctes

S13
THÉORIE**Textures OpenGL — Fondements**

- glGenTextures, glBindTexture, glTexImage2D : upload CPU→GPU
- Formats internes : GL_RGBA8, GL_RGB8, GL_R8, GL_RGBA16F
- Mipmap : glGenerateMipmap, niveaux LOD, calcul automatique
- Filtering : GL_NEAREST, GL_LINEAR, GL_LINEAR_MIPMAP_LINEAR (trilinear)
- Wrap modes : GL_REPEAT, GL_CLAMP_TO_EDGE, GL_MIRRORED_REPEAT
- Texture Units : glActiveTexture(GL_TEXTURE0+N), glUniform1i

S14
THÉORIE**Sampler Objects & Texture Arrays**

- Sampler Objects (GL 3.3) : dissocier paramètres de sampling de la texture
- glGenSamplers, glSamplerParameteri, glBindSampler
- Texture Arrays (GL_TEXTURE_2D_ARRAY) : plusieurs textures en un seul objet
- Cubemaps (GL_TEXTURE_CUBE_MAP) : environnement 360°, skybox
- Anisotropic filtering (EXT_texture_filter_anisotropic)
- Texture compression : GL_COMPRESSED_RGBA_S3TC_DXT5_EXT

S15
TP**TP — Texturing Avancé**

- Charger des textures PNG/JPG via stb_image (autorisé)
- Classe Texture2D : encapsule glGenTextures, upload, sampler
- Multi-texturing : mixer 2 textures dans un fragment shader
- Sprite sheet animé : UV offset dans le shader via uniforms
- Cubemap : skybox avec texture cubemap

Livrable : Scène 3D avec textures, multi-texturing et skybox cubemap

S16
THÉORIE**Caméra FPS & Arcball**

- Caméra FPS : yaw/pitch/roll via euler angles, direction vector
- Entrée souris : NkMouseMoveEvent → delta → rotation caméra
- lookAt(eye, center, up) : construction de la view matrix depuis NKMath
- Zoom : modifier le FOV (perspective), ou scale (ortho)
- Caméra Arcball : rotation autour d'un point d'intérêt
- NkQuaternion : interpolation SLERP pour les transitions de caméra

S17
THÉORIE**Frustum Culling 3D**

- Frustum : 6 plans (near, far, left, right, top, bottom)
- Extraction des plans depuis la matrice MVP (Gribb-Hartmann)
- AABB vs Frustum : test d'appartenance, rejeter les objets non visibles
- Bounding sphere vs frustum : test plus rapide
- Occlusion culling : concept, HZB (Hierarchical Z-Buffer)

S18
TP**TP — Caméra Interactive**

- NkQuaternion dans NKMath (slerp, toMat4)
- Caméra FPS : WASD + souris via NkEventSystem
- Caméra Arcball : drag pour orbiter autour d'une cible
- Frustum culling : afficher le nombre d'objets cullés dans le HUD
- Scène de 100+ objets avec frustum culling actif

Livrable : Scène 3D navigable FPS + arcball avec frustum culling mesuré

S19
THÉORIE**Modèle de Phong**

- Composantes : Ambient + Diffuse + Spéculaire
- Normales : attribut vertex, interpolation (Gouraud vs Phong shading)
- Normal matrix : $\text{transpose}(\text{inverse}(\text{mat3}(\text{Model})))$ — pourquoi ça compte
- Types de lumières : directionnelle, point, spot
- Atténuation : constante, linéaire, quadratique ($1/(k_c + k_l*d + k_q*d^2)$)
- Blinn-Phong : half-vector — plus rapide et plus réaliste

S20
THÉORIE**Physically Based Rendering — Introduction**

- Motivations PBR : éclairage physiquement correct, artistes-friendly
- Microfacet theory : roughness, metallic, F0 (Fresnel à incidence 0)
- Cook-Torrance BRDF : D (distribution), G (géométrie), F (Fresnel)
- Textures PBR : albedo, normal map, metallic, roughness, AO
- Gamma correction et HDR : espace linéaire de calcul, tonemapping

S21
TP**TP — Éclairage Phong & PBR Basique**

- Phong shading complet en GLSL : ambient + diffuse + spéculaire
- Multiple lumières (point lights) dans un tableau uniform
- Normal maps : TBN matrix, tangent space shading
- PBR basique : albedo + roughness + metallic + un point light
- Gamma correction : lineariser les textures sRGB, encoder la sortie

Livrable : Scène 3D avec Phong + PBR side-by-side, multiple lumières

S22
THÉORIE**Scene Graph 3D**

- Node : NkTransform3D + parent + enfants + mesh + material
- Traversal récursif : calculer le world matrix = parent_world * local
- Scene Graph vs ECS : avantages respectifs
- Dirty flag : ne recalculer que les branches modifiées
- Material system : shader + uniforms + textures liés à un mesh

S23
THÉORIE**Instancing & Draw Indirects**

- Instancing : dessiner N copies d'un mesh en un seul draw call
- glDrawArraysInstanced / glDrawElementsInstanced
- gl_InstanceID dans le vertex shader : indexer dans un tableau
- Instanced attributes : glVertexAttribDivisor(1) — par instance
- SSBO (Shader Storage Buffer Object) : matrices par instance
- Indirect draw calls : glDrawElementsIndirect

S24
TP**TP — Forêt Instanciée**

- Scene Graph avec Node, traversal, world matrix
- 1000 arbres instanciés en un seul glDrawElementsInstanced
- Matrices par instance dans un VBO avec glVertexAttribDivisor
- Frustum culling sur CPU avant l'envoi des matrices
- Compteur de draw calls affiché dans le HUD

Livrable : Forêt de 1000 arbres à 60 FPS avec un seul draw call

S25
THÉORIE**Framebuffer Objects (FBO)**

- FBO : redirectionner le rendu vers une texture
- Attachments : GL_COLOR_ATTACHMENT0, GL_DEPTH_ATTACHMENT, GL_STENCIL_ATTACHMENT
- Renderbuffer Objects (RBO) : attachement optimisé (pas samplable)
- glCheckFramebufferStatus : vérifier la complétude
- Multi-sample FBO : MSAA, glBlitFramebuffer pour resolve
- MRT (Multiple Render Targets) : écrire dans plusieurs textures simultanément

S26
THÉORIE**Effets Post-Processing**

- Screen-space quad : rendu de la scène dans un FBO, post-process en FS
- Effets : blur (Gaussien), bloom (bright-pass + blur), FXAA, vignette
- HDR + Tonemapping : rendu en GL_RGBA16F, tone map dans le final pass
- Color grading : LUT (Look-Up Table) 3D
- SSAO (Screen Space Ambient Occlusion) : occlusion approximée

S27
TP**TP — Pipeline de Post-Processing**

- FBO avec color (GL_RGBA16F) + depth/stencil attachments
- Passes : geometry pass → bloom bright-pass → blur → composite
- HDR tonemapping : Reinhard ou ACES filmic
- Effet SSAO basique
- UI overlay : dessiner HUD après post-processing (FBO séparé)

Livable : Pipeline multi-pass : HDR + Bloom + SSAO + Tonemapping

S28
THÉORIE**Geometry Shader**

- Geometry Shader : étape entre VS et FS, peut émettre/supprimer des primitives
- Primitives d'entrée : points, lines, triangles, adjacency
- EmitVertex(), EndPrimitive() : construire des primitives
- Cas d'usage : normales debug (visualiser les vecteurs normaux)
- Billboards : orienter un quad toujours face à la caméra
- Explode shader : déplacer les faces le long de leurs normales

S29
THÉORIE**Tessellation — Introduction**

- Tessellation Control Shader (TCS) : niveaux de subdivision
- Tessellation Evaluation Shader (TES) : interpolation des vertices
- Modes : triangles, quads, isolines
- LOD dynamique : subdiviser plus près de la caméra
- Displacement mapping : déplacer les vertices selon une heightmap

S30
TP**TP — Geometry & Tessellation**

- Debug overlay de normales via geometry shader
- Billboards GPU : arbres/particules toujours face caméra
- Terrain subdivisé avec TCS/TES + displacement map
- Grass rendering via geometry shader (générer des brins)

Livable : Terrain subdivisé + herbe GPU + normales visualisées

S31
THÉORIE**Compute Shaders**

- Compute Shader (GL 4.3) : exécuter du calcul arbitraire sur le GPU
- Work groups : local_size_x/y/z, glDispatchCompute
- SSBO : lecture/écriture de buffers arbitraires depuis un compute shader
- Image Load/Store : accès direct aux pixels d'une texture (imageLoad/imageStore)
- Memory barriers : glMemoryBarrier — synchroniser CPU/GPU et shader/shader
- Atomic operations : atomicAdd, atomicMin sur des SSBO

S32
THÉORIE**Cas d'Usage GPGPU**

- Simulation de particules GPU : position + vitesse dans des SSBO, update en compute
- Frustum culling GPU : tester les bounding spheres en compute, écrire les draw calls indirects
- Réduction parallèle : somme, max, histogramme sur de grandes données
- Prefix sum (scan) : algorithme parallèle fondamental
- Fluid simulation 2D : grille de vitesse et pression en compute

S33
TP**TP — Simulation de Particules GPU**

- 1 million de particules : position/vitesse dans des SSBO
- Compute shader : intégration Euler + collision sphère
- Rendu : gl_PointSize dans le vertex shader, billboards
- GPU culling : compute → draw indirect en un frame
- Comparaison performance CPU vs GPU

Livable : 1M particules simulées et rendues entièrement sur GPU

S34
THÉORIE**Shadow Mapping**

- Shadow map : rendre la scène depuis la lumière dans un depth FBO
- Comparaison de profondeur : shadow test dans le fragment shader
- Perspective / orthographic shadow map selon le type de lumière
- Artefacts : shadow acne (depth bias), peter panning, aliasing
- PCF (Percentage Closer Filtering) : adoucir les ombres
- Cascaded Shadow Maps (CSM) : plusieurs niveaux de résolution

S35
THÉORIE**PBR Complet & IBL**

- Cook-Torrance BRDF complet : D (GGX/Trowbridge-Reitz), G (Smith), F (Schlick)
- Image-Based Lighting (IBL) : environnement comme source de lumière
- Irradiance map : intégrale diffuse pré-calculée (convolution cubemap)
- Pre-filtered environment map : spéculaire selon roughness
- BRDF LUT : intégrale spéculaire pré-calculée (split-sum approx)
- Emission, clearcoat, sheen : extensions du modèle PBR

S36
TP**TP — Scène PBR + Ombres**

- Shadow map avec PCF (noyau 3×3)
- PBR complet : albedo + normal + metallic + roughness + AO + emission
- IBL : charger un HDR environment, générer irradiance + prefilter + LUT
- Tone mapping ACES filmic + gamma correction
- Scène de démonstration : sphères PBR avec différents matériaux

Livrable : Scène PBR réaliste avec ombres douces et IBL

Description

Les étudiants livrent un moteur de rendu 3D OpenGL complet, avec sa démo interactive et son rapport technique. Individuel ou binôme.

Exigences Minimales

1. Contexte OpenGL 3.3+ via NKWindow, Jenga multi-plateforme
2. NKMath complète : NkVec3/4f, NkMat4x4, NkQuaternion, NkTransform3D
3. Pipeline : VAO/VBO/EBO, shaders GLSL compilés, uniform system
4. Texturing : texture 2D + normal map + PBR textures
5. Éclairage Phong ou PBR avec au moins 3 types de lumières
6. Caméra FPS ou arcball interactive (NkEventSystem)
7. FBO avec au moins 2 effets post-processing
8. Scene graph ou ECS avec frustum culling
9. 60 FPS stable en 1280×720

Bonus

- Shadow mapping avec PCF
- IBL complet (irradiance + prefilter + BRDF LUT)
- Instancing GPU + indirect draw
- Compute shader (simulation ou culling)
- Geometry shader (grass, billboards)

Grille d'Évaluation

Critère	Points	Vérification
Pipeline GL fonctionnel (VAO/VBO/EBO + shaders)	/20	drawCall() démontré
NKMath complète (6 types documentés)	/15	Chaque type testé
Textures + normal maps	/10	Scène texturée
Éclairage Phong ou PBR	/15	3 lumières minimum
Caméra interactive (NkEventSystem)	/10	Démo en direct

Post-processing (FBO, 2 effets)	/10	Visible à la démo
Scene graph + frustum culling	/10	Compteur affiché
Rapport technique (10 pages)	/10	Livré en PDF
BONUS : shadows + IBL + compute	+10	Benchmark documenté

Évaluation Globale

Critère	Poids	Modalité
TPs hebdomadaires (code + démo + NKMath)	10 %	Continu
Examens théoriques SN	40 %	
Projet final — code + NKMath	20 %	
Soutenance + rapport	20 %	
CC	20%	